

# Machine Learning

---

Jaeyoung Yoon

March 18, 2022

Seoul National University  
jyoung924@snu.ac.kr

# What is Machine Learning?

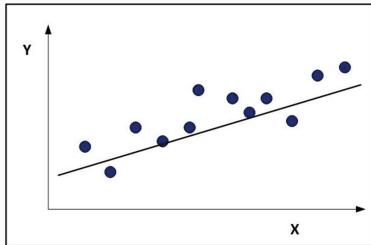
**Machine Learning** : the field of study that gives computers the ability to learn **without** being explicitly **programmed**.

- Supervised Learning
- Unsupervised Learning
- Reinforcement learning

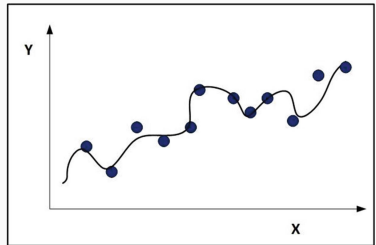


# Overfitting, Underfitting

We want to construct a prediction model for the followings:



Underfitting



Overfitting

# Bias, Variance

**Bias(편향)** : the expected error created by using a model to approximate a real world.

$$\text{Bias} [\hat{f}(x)] = \mathbb{E} [\hat{f}(x) - f(x)]$$

High Bias  $\iff$  Underfitting

**Variance(분산)** : the amount our predicted values would change if we had a different training dataset.

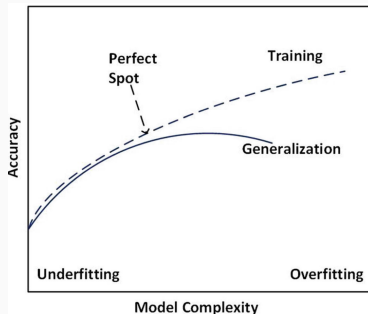
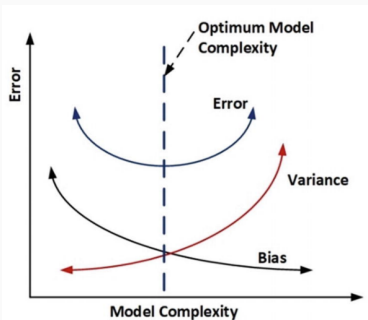
$$\text{Var} [\hat{f}(x)] = \mathbb{E} [\hat{f}(x)^2] - \mathbb{E} [\hat{f}(x)]^2$$

High Variance  $\iff$  Overfitting

$f$  : objective function,  $\hat{f}$  : prediction function.

# Bias-Variance Tradeoff

: the property of a model that the variance of the parameter estimated across samples can be reduced by increasing the bias in the estimated parameters.



# Accuracy, Regularization

- Accuracy

$$\text{accuracy} = \frac{\text{number of correctly classified cases}}{\text{total number of cases}}$$
$$\text{error} = 1 - \text{accuracy}$$

- Regularization

$$C(\tau) = \text{regularizer} + \text{loss}(\tau) = \alpha(\lambda) + \text{loss}(\tau),$$

$C$ : cost function,  $\alpha \geq 0$ : regularizer or penalty function

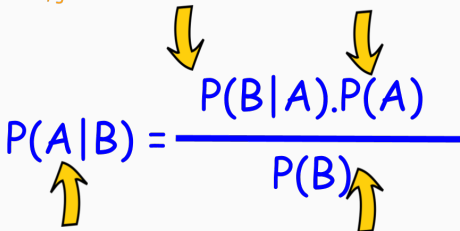
# Bayesian Learning

## LIKELIHOOD

The probability of "B" being True, given "A" is True

## PRIOR

The probability "A" being True. This is the knowledge.


$$P(A|B) = \frac{P(B|A).P(A)}{P(B)}$$

## POSTERIOR

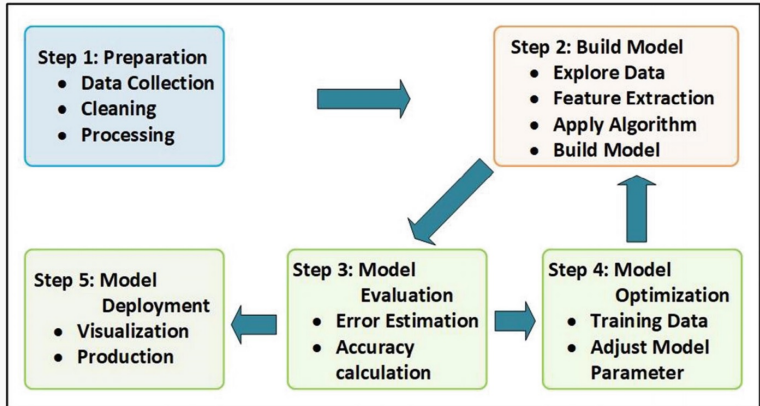
The probability of "A" being True, given "B" is True

## MARGINALIZATION

The probability "B" being True.

A: hypothesis whose probability may be affected by 'evidence',  
B: evidence.

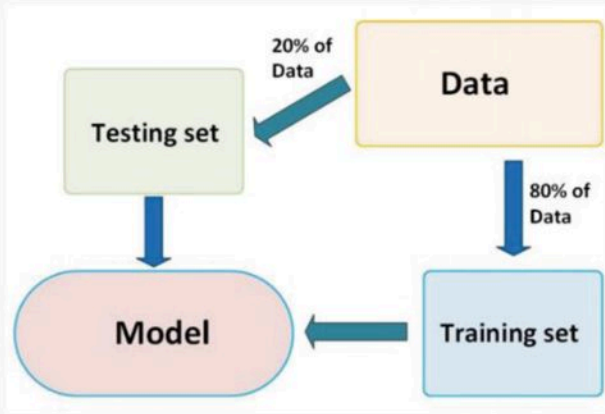
# Machine Learning Workflow



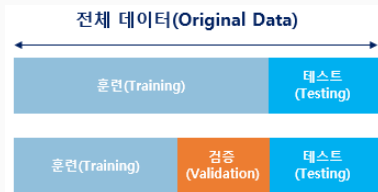
Machine Learning Workflow

# Validation of Models

## The Hold-Out Method



# Validation of Models: The Cross-Validation Method



Note) Testing set is **not** used in learning. Only once used when we measure the ability.

- k-fold cross-validation

| Runs | Splits |       |       |       |       |       | Accuracy |
|------|--------|-------|-------|-------|-------|-------|----------|
| 1    | Test   | Train | Train | Train | Train | Train | 80       |
| 2    | Train  | Test  | Train | Train | Train | Train | 82       |
| 3    | Train  | Train | Test  | Train | Train | Train | 88       |
| 4    | Train  | Train | Train | Test  | Train | Train | 90       |
| 5    | Train  | Train | Train | Train | Test  | Train | 85       |
| 6    | Train  | Train | Train | Train | Train | Test  | 92       |

6-fold cross validation

## Example(Python): $k$ -fold cross validation

```
# Import Libraries
import numpy as np
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn import svm # import (SVM) for classification
from sklearn import datasets
iris = datasets.load_iris() # load the data
# Data split into train/test sets with 25% for testing
X_train, X_test, y_train, y_test = train_test_split(iris.data, iris.target, test_size=0.25,
                                                    random_state=0)
# Train data for Linear Support Vector Classifier (SVC) model for prediction
clf = svm.SVC(kernel='linear', C=1).fit(X_train, y_train)
# Performance measurement with the test data
clf.score(X_test, y_test)
```

✓ 0.1s

0.9736842105263158

## Example(Python): $k$ -fold cross validation

```
svc = svm.SVC(kernel='linear', C=1).fit(X_train, y_train)
# cross_val_score a model, the data set, and the number of folds:
cvs = cross_val_score(svc, iris.data, iris.target, cv=5)
# Print accuracy of each fold:
print(cvs)
# Mean accuracy of all 5 folds:
print(cvs.mean())
```

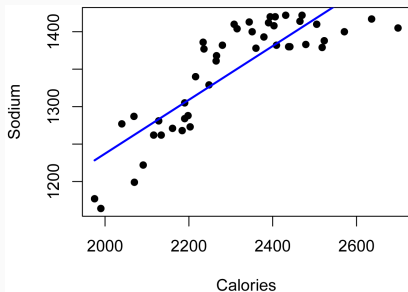
✓ 0.3s

```
[0.96666667 1.          0.96666667 0.96666667 1.          ]
0.98000000000000001
```

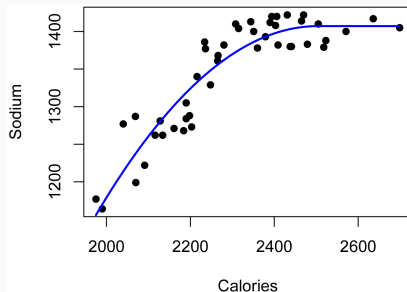
5-fold cross validation

# Regression

: estimating the relationships among variables



Linear Regression



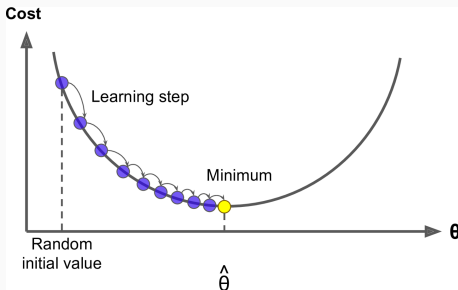
Nonlinear Regression

# Gradient Descent

: iterative technique to find a minimum point of the cost function( $C$ ).

$$\theta^{(n+1)} = \theta^{(n)} - \alpha_n \nabla C(\theta^{(n)}),$$

where  $\theta^{(n)} \in \mathbb{R}^N$  is a model parameter and  $\alpha_n$  is a learning rate at  $n$ -th iteration.



# Stochastic gradient descent (SGD)

SGD:

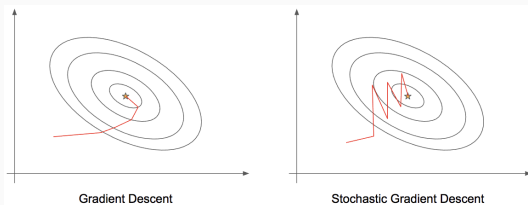
$$i(n) \sim \text{Uniform}\{1, \dots, N\}$$
$$\theta^{(n+1)} = \theta^{(n)} - \alpha_n \nabla C_{i(n)}(\theta^{(n)}),$$

where  $C = \sum_{i=1}^N C_i$ .

$\nabla C_{i(n)}(\theta^{(n)})$  is a stochastic gradient of  $C$  at  $\theta^{(n)}$ , i.e.,

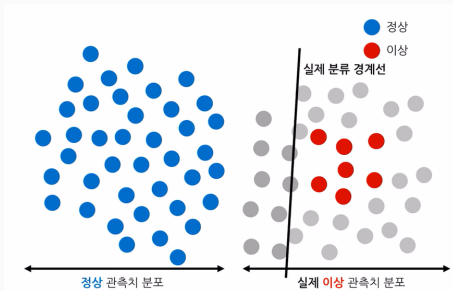
$$\mathbb{E}[\nabla C_{i(n)}(\theta^{(n)})] = \nabla \mathbb{E}[C_{i(n)}(\theta^{(n)})] = \nabla C(\theta^{(n)}).$$

When size of the data  $N$  is large, SGD is often more effective than GD.



## Classification

- Binary classifiers
- Multi-class classifiers
- Multi-label classifiers
- Imbalanced classifiers



# Confusion Matrix

TP : TruePositive, FP : FalsePositive,

TN : TrueNegative, FN : FalseNegative

|                 | Classified as <b>YES</b> | Classified as <b>NO</b> |
|-----------------|--------------------------|-------------------------|
| True <b>YES</b> | No. of TPs               | No. of FNs              |
| True <b>NO</b>  | No. of FPs               | No. of TNs              |

$$\text{accuracy} = \frac{\text{number of correctly classified cases}}{\text{total number of cases}} = \frac{TPs + TNs}{\text{total number of cases}}$$

$$\text{precision(정밀도)} = \frac{TPs}{TPs + FPs}$$

$$\text{recall(재현율)} = \frac{TPs}{TPs + FNs}$$

f-score(f-measure) : the harmonic mean of *precision* and *recall*.

# Supervised Learning

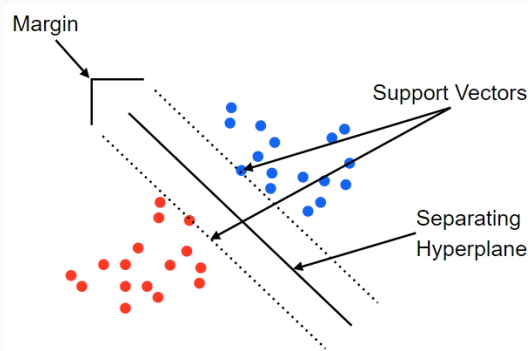
: the machine learning task of learning a function that maps an input to an output based labeled data.

| Supervised Learning Methods | Example                                                               | Algorithms                                  |
|-----------------------------|-----------------------------------------------------------------------|---------------------------------------------|
| Regression                  | Stock price prediction, House price prediction                        | Linear regression, ridge, lasso, polynomial |
| Classification              | Spam: Fraud/Not Fraud, Cancer/Not Cancer                              | Logistic regression, SVM, KNN               |
| Decision Tress              | Predict stock prices (regression), classification (credit card fraud) | Decision trees, random forest               |

## Examples of Supervised Learning Algorithms

# Support-Vector Machines (SVM)

: constructs a (set of) **hyperplanes** in a high-dimensional space, which can be used for classification, regression, so on.



# Example(Python): Support-Vector Machines(SVM)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn import svm, datasets
```

```
def make_meshgrid(x, y, h=0.02):
    """Create a mesh of points to plot in
```

Parameters

x: data to base x-axis meshgrid on  
y: data to base y-axis meshgrid on  
h: stepsize for meshgrid, optional

Returns

xx, yy : ndarray

xx\_min, xx\_max = x.min() - 1, x.max() + 1  
yy\_min, yy\_max = y.min() - 1, y.max() + 1  
xx, yy = np.meshgrid(np.arange(xx\_min, xx\_max, h), np.arange(yy\_min, yy\_max, h))  
return xx, yy

```
def plot_contours(ax, clf, xx, yy, **params):
    """Plot the decision boundaries for a classifier.
```

Parameters

ax: matplotlib axes object  
clf: a classifier  
xx: meshgrid ndarray

```
xx: meshgrid ndarray
yy: meshgrid ndarray
params: dictionary of params to pass to contourf, optional
"""
```

```
Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)
out = ax.contourf(xx, yy, Z, **params)
return out
```

# Import some data to play with

iris = datasets.load\_iris()

# Take the first two features. We could avoid this by using a two-dim dataset

X = iris.data[:, :2]

y = iris.target

# We create an instance of SVM and fit out data. We do not scale our

# data since we want to plot the support vectors

C = 1.0 # SVM regularization parameter

models = {

svm.SVC(kernel="linear", C=C),

svm.LinearSVC(C=C, max\_iter=10000),

svm.SVC(kernel="rbf", gamma=0.7, C=C),

svm.SVC(kernel="poly", degree=3, gamma="auto", C=C),

}

models = (clf.fit(X, y) for clf in models)

# title for the plots

titles = {

"SVC with linear kernel",

"LinearSVC (linear kernel)",

"SVC with RBF kernel",

"SVC with polynomial (degree 3) kernel",

}

# Example(Python): Support-Vector Machines(SVM)

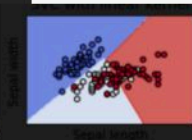
```
# Set-up 2x2 grid for plotting.
fig, sub = plt.subplots(2, 2)
plt.subplots_adjust(wspace=0.4, hspace=0.4)

X0, X1 = X[:, 0], X[:, 1]
xx, yy = make_meshgrid(X0, X1)

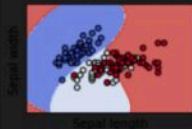
for clf, title, ax in zip(models, titles, sub.flatten()):
    plot_contours(ax, clf, xx, yy, cmap=plt.cm.coolwarm, alpha=0.8)
    ax.scatter(X0, X1, c=y, cmap=plt.cm.coolwarm, s=20, edgecolors=
    ax.set_xlim(xx.min(), xx.max())
    ax.set_ylim(yy.min(), yy.max())
    ax.set_xlabel("Sepal length")
    ax.set_ylabel("Sepal width")
    ax.set_xticks(())
    ax.set_yticks(())
    ax.set_title(title)

plt.show()
```

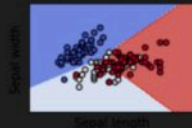
✓ 0.8s



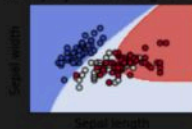
SVC with RBF kernel



SVC with polynomial (degree 3) kernel



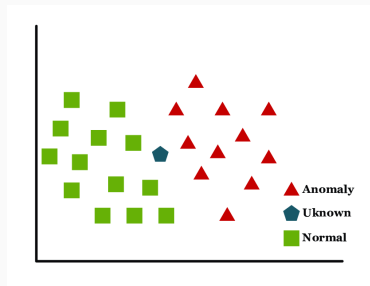
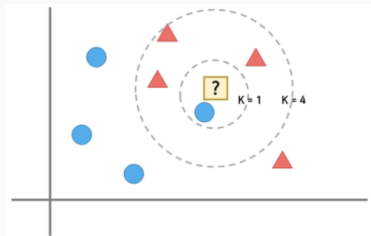
SVC with polynomial (degree 3) kernel



SVC with polynomial (degree 3) kernel

# $k$ -Nearest Neighbors (KNN)

: classify the data depending on the nearest  $k$  labeled data.



pros) easy and fast

cons) a single error of data can occur poor classification error.

# Error and Loss function

$x$  : actual value,  $\hat{x}$  : predicted values.

$$E_{SSE} = \frac{1}{2} \sum_n \left( x^{(n)} - \hat{x}^{(n)} \right)^2, \quad \text{sum of squared errors}$$

$$E_{MSE} = \frac{1}{n} \sum_n \left( x^{(n)} - \hat{x}^{(n)} \right)^2, \quad \text{mean squared error}$$

$$E_{MAE} = \frac{1}{n} \sum_n |x^{(n)} - \hat{x}^{(n)}|, \quad \text{mean absolute error}$$

$$E_{BCE} = -\frac{1}{n} \sum_n \left[ x^{(n)} \log \hat{x}^{(n)} + (1 - x^{(n)}) \log (1 - \hat{x}^{(n)}) \right],$$

Binary cross entropy

# Unsupervised Learning

Goal : learn the structure of the dataset which is unlabeled.

| Unsupervised Learning Methods | Example                                            | Algorithms                                  |
|-------------------------------|----------------------------------------------------|---------------------------------------------|
| Clustering                    | Cluster virus data, visitor groups at a conference | Hierarchical clustering, k-means clustering |
| Dimensionality reduction      | Reduction of dimensions in data                    | Principal component analysis (PCA)          |

Examples of Unsupervised Learning Algorithms

# *k*-Means Clustering

Given an initial set of  $k$  means  $m_1^{(1)}, \dots, m_k^{(1)}$ ,

## 1. (Assignment step)

Assign each observation to the cluster with the nearest mean, i.e.,

$$S_i^{(t)} = \{x_p : \|x_p - m_i^{(t)}\|^2 \leq \|x_p - m_j^{(t)}\|^2, \forall j = 1, \dots, k\},$$

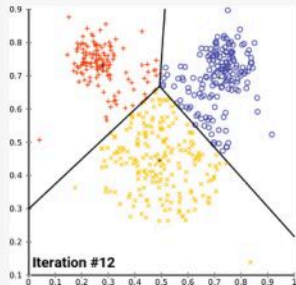
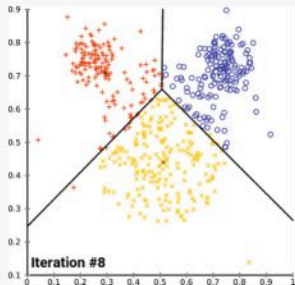
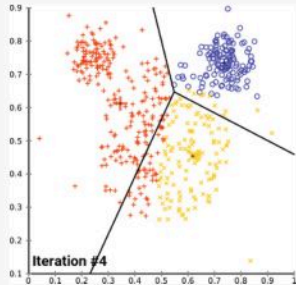
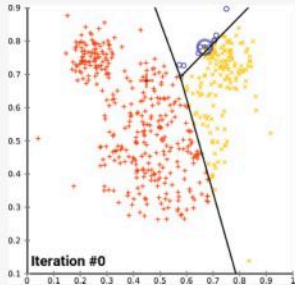
where each  $x_p$  is assigned to exactly one  $S^{(t)}$ .

## 2. (Update step)

Recalculate means by observations assigned to each cluster.

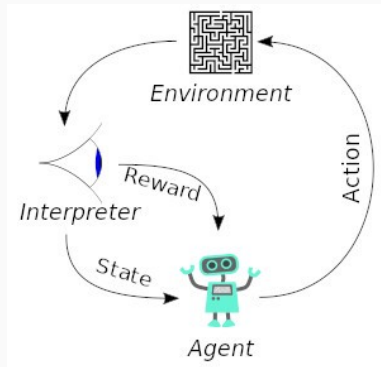
$$m_i^{(t+1)} = \frac{1}{|S_i^{(t)}|} \sum_{x_j \in S_i^{(t)}} x_j.$$

# Example: $k$ -Means Clustering



# Reinforcement Learning

: Agents are trained on a **reward** and punishment mechanism.



e.g.) Self-driving cars, personalised product recommendation system, etc.